

Final Project Report: LLM-Driven Robotic Arm Tic-Tac-Toe System

Team Number: Team 13

Team Members:

- Song Zhiyuan (225040265) – LLM/VLM Integration & Prompt Engineering
- Chen Shaokai (225040280) – Robot Control & ROS MoveIt! Programming
- Li Yutian (225040243) – Hardware Setup, Calibration & Vision System

Project Repository:

<https://github.com/837944459-star/AIR-5021-2026--Team13-FinalProject>

1. Project Overview

Goal: Our robot autonomously plays Tic-Tac-Toe against a human opponent. The system uses a Sagittarius SGR532 6-DOF robotic arm to observe the board state through a camera(usb-cam), reason about the optimal move using a Large Language Model, and physically place game pieces on a 3×3 grid. The robot serves as the "O" player (deep-blue cubes) while the human plays as "X" (yellow cubes). The full game loop—perceive, reason, act—runs autonomously without human intervention after initial calibration.

Innovation & Intelligence: This project integrates two distinct AI capabilities through Alibaba Cloud's Qwen model family, creating a truly intelligent robotic system:

- **Vision-Language Model for Board Perception:** qwen-vl-max, a Vision-Language Model (VLM), replaces traditional computer-vision pipelines (e.g., YOLO, color thresholding). Instead of training object detectors, we use carefully engineered prompts that describe the physical board setup (15 cm × 15 cm cardboard, black pen grid lines, yellow and deep-blue cubes). The VLM directly outputs a structured JSON board state, dramatically reducing development effort and enabling flexible adaptation to new board configurations through prompt changes alone.
- **LLM for Game Strategy:** qwen-plus, a text LLM, generates strategic moves. Rather than hardcoding minimax or alpha-beta pruning, we instruct the LLM with game rules and a priority hierarchy. The LLM also returns JSON with the chosen position and its reasoning, providing human-interpretable decision explanations.

Hardware Setup: The system uses a Sagittarius SGR532 6-DOF robotic arm controlled via ROS Noetic with MoveIt! for motion planning. A USB webcam mounted above the workspace captures board images for the VLM pipeline. The game board is a 15 cm × 15 cm cardboard sheet with a hand-drawn 3×3 grid (5 cm cell spacing). Game pieces are 2.4 cm foam cubes—deep-blue for the robot, yellow for the human—with a storage area located 3 cm to the right of the board's top-right corner. All computation runs on a local Ubuntu machine,

and LLM/VLM inference is performed via the Alibaba Cloud DashScope API.

2. Design & Implementation

2.1 System Architecture

The system follows a modular pipeline architecture with four layers:

```
Camera → VLM (qwen-vl-max) → Board State → LLM (qwen-plus) → Move Decision → MoveIt!  
Control → Robotic Arm
```

Each layer is implemented as an independent Python class, enabling separate testing and iteration:

- **Perception:** TicTacToeVision – VLM-based board state detection
- **Reasoning:** TicTacToeLogic – LLM-based move generation with fallback
- **Execution:** TicTacToeControl – MoveIt!-based arm control with safety protocols
- **Sensing:** CameraCapture – ROS image subscription and capture
- **Orchestration:** TicTacToeGame – Orchestrates the full game loop

2.2 Task Planning (The "Brain")

The LLM receives a text prompt encoding the current board state as a human-readable grid (using ".", "O", "X" symbols), along with game rules and a strategy priority hierarchy. The system prompt instructs the model to: (1) check for an immediate winning move, (2) block the opponent's winning move, (3) occupy the center or corner positions strategically, and (4) select the first available cell as a last resort. The LLM returns a JSON response with the chosen row, column, and a human-readable reason string.

A robust fallback mechanism ensures reliability: if the LLM returns an illegal move (occupied cell, out-of-range coordinates) or the API call fails entirely, the system iterates through empty positions and selects the first valid one. This guarantees that the game always continues even when the LLM produces suboptimal or erroneous outputs.

The VLM prompt engineering is equally critical. Rather than sending raw images with minimal context, the prompt explicitly describes the physical setup—board dimensions, grid line color, piece colors and sizes—and requests a structured JSON response {"board": [[0,0,0],[0,1,0],[2,0,0]]}. This approach eliminates the need for traditional object detection training and allows rapid reconfiguration (e.g., changing piece colors) by simply updating the prompt text.

2.3 Robot Skills (The "Actions")

The robotic arm implements the following core skills:

- **Grasp from Storage:** Picks up a deep-blue cube from the designated storage area using

a carefully sequenced motion: approach at safe height → descend to hover → descend to grasp height → close gripper → ascend to safe height. All vertical movements use Cartesian path planning to ensure straight-line motion and avoid collisions with existing pieces.

- **Place Piece on Board:** Transports the grasped piece to the target grid cell and releases it: move horizontally at safe height (10 cm above board) → descend to hover height (6 cm) → descend to place height (1.2 cm) → open gripper → ascend to safe height.
- **Observe Board:** Moves the arm to a calibrated observation position above the board, allowing the camera to capture a clear, unobstructed view of the entire grid for VLM analysis.
- **Home/Sleep Positions:** Returns the arm to its home configuration (upright position) and sleep configuration (folded compact position).
- **Gripper Control:** Opens and closes the parallel-jaw gripper with controlled timing for mechanical settling.

2.4 Motion Planning Strategy

A key design decision was the hybrid motion planning strategy that combines two approaches:

- **Joint-Angle Planning (Primary):** The primary method loads pre-calibrated joint angles for each grid cell from a YAML configuration file. This completely bypasses inverse kinematics, eliminating IK failures and guaranteeing that every target position is reachable. During calibration, the user need to use RVIZ(demo_true.launch) and control the arm to each cell and records both Cartesian coordinates and joint angles.
- **Cartesian Path Planning (Vertical):** Vertical movements (ascending and descending over the board) always use Cartesian path planning via `compute_cartesian_path`. This ensures the gripper moves in perfectly straight vertical lines, preventing it from sweeping horizontally through existing pieces.

A three-layer height stratification system prevents collisions:

- **Layer 1:** `SAFE_HEIGHT` (10 cm above board): used for all horizontal transit between cells
- **Layer 2:** `HOVER_HEIGHT` (6 cm above board): intermediate stop before descending to grasp or place
- **Layer 3:** `GRASP/PLACE_HEIGHT` (1.2 cm above board): actual contact height for piece manipulation

2.5 Calibration System

The system includes an interactive calibration tool (`calibrate_grid.py`) that allows the user to

manually position the arm at each grid cell and record both Cartesian coordinates and joint angles. The tool validates calibration data by checking for missing cells, Z-value consistency (tolerance < 1 cm), grid spacing (expected 5 cm between cells), and joint data presence. Results are saved to a YAML file that the control module loads at runtime.

3. Experiments & Results

3.1 Test Environment

All testing was conducted on the lab's computer. The Sagittarius SGR532 arm was mounted on a stable base with the game board (15 cm $\times$ 15 cm cardboard with hand-drawn grid) placed within the arm's reachable workspace. A USB webcam was mounted on a fixed stand approximately 30 cm above the board, providing a top-down view. The Ubuntu workstation running ROS Noetic was connected to both the arm and the camera. LLM/VLM inference was performed via cloud API calls to Alibaba Cloud DashScope.

3.2 VLM Board Detection Accuracy

The VLM-based board detection was tested across multiple game states with varying numbers of pieces on the board. Key findings:

- The VLM achieves high accuracy when the board has 0–4 pieces, correctly identifying all piece positions and colors.
- Accuracy decreases slightly with 5+ pieces, particularly when pieces are adjacent, as the model occasionally confuses the grid cell assignment for closely placed cubes.
- Lighting conditions significantly affect detection quality. Even, diffused lighting yields the best results; harsh shadows or reflections cause false detections.
- The prompt engineering approach proved highly effective—changing the piece color description in the prompt was sufficient to adapt to different colored pieces without any model retraining.

3.3 LLM Move Generation Quality

The LLM strategy was evaluated on its ability to make optimal moves in common game scenarios:

- Winning move detection: The LLM correctly identifies immediate winning moves in $>95\%$ of tested scenarios.
- Blocking opponent wins: The LLM successfully blocks opponent winning moves in $>90\%$ of cases, occasionally missing diagonal threats.
- Strategic play: When no immediate win or block is available, the LLM preferentially selects center and corner positions, consistent with sound Tic-Tac-Toe strategy.
- The fallback mechanism (first empty cell) was invoked in approximately 5% of moves,

primarily due to API timeouts or malformed JSON responses.

3.4 Motion Execution Reliability

The hybrid joint-angle + Cartesian motion strategy proved highly reliable:

- Joint-angle planning achieved a 100% planning success rate for horizontal positioning, as all target joint configurations were pre-validated during calibration.
- Cartesian path planning for vertical movements achieved >95% success rate, with failures occurring only when the arm was near joint limits.
- End-effector positioning accuracy was within 5 mm of the calibrated target, sufficient for placing 2.4 cm cubes on 5 cm grid cells.
- Piece placement success rate (piece remains standing after gripper release) was approximately 75%, with failures primarily due to small friction of grippers and small cubes.

3.5 End-to-End Game Performance

Complete Tic-Tac-Toe games were played with the robot taking the first move. The system successfully completed full games (from first move to win/loss/draw detection) in the majority of trials. A single game cycle (robot turn: observe → reason → act) takes approximately 30–45 seconds, with the majority of time consumed by VLM/LLM API calls (~5–10 seconds each) and motion execution (~15–20 seconds).

3.6 Challenges & Solutions

- **IK Planning Failures:** Early in development, we relied on MoveIt! inverse kinematics for all positioning, which frequently failed (~40% of attempts) due to the arm's workspace constraints. Solution: We switched to a joint-angle-first strategy, pre-calibrating all target positions during a manual setup phase. This eliminated IK failures entirely for horizontal positioning.
- **Collision with Existing Pieces:** The gripper occasionally knocked over existing pieces while moving horizontally across the board. Solution: We implemented the three-layer height stratification system (safe → hover → grasp/place), ensuring all horizontal transit occurs at 10 cm above the board, well above any piece height.
- **VLM Detection Unreliability:** The VLM sometimes returned malformed JSON or incorrectly parsed board states, especially under poor lighting. Solution: We added a board state validation function that checks dimensions (3×3), valid cell values (0/1/2), and consistency with the previous state. Invalid detections trigger a re-capture and re-detection cycle.
- **API Latency:** Cloud API latency introduced 5–10 second delays per LLM/VLM call, slowing the game pace. Solution: We implemented asynchronous processing where

possible and reduced the prompt length to minimize token generation time. We also added a manual human-move input mode as a fallback when automatic detection is too slow.

- **Calibration Drift:** Slight drift in arm positioning over time caused pieces to be placed off-center. Solution: We implemented a re-calibration protocol and added position tolerance checks before each game session.

4. Photo & Video Documentation

Final Demo: The final demo shows the robot successfully completing a Tic-Tac-Toe game against a human opponent. The video documentation includes:

- System setup: arm, camera, board, and pieces in the lab environment
- Calibration process: manually positioning the arm to record grid cell coordinates
- Full game play: the robot observes the board via VLM, reasons about its move via LLM, and executes the placement via MoveIt!
- Close-up of piece grasping from storage area and precise placement on the target grid cell
- Game completion: win/loss/draw detection and arm returning to home position

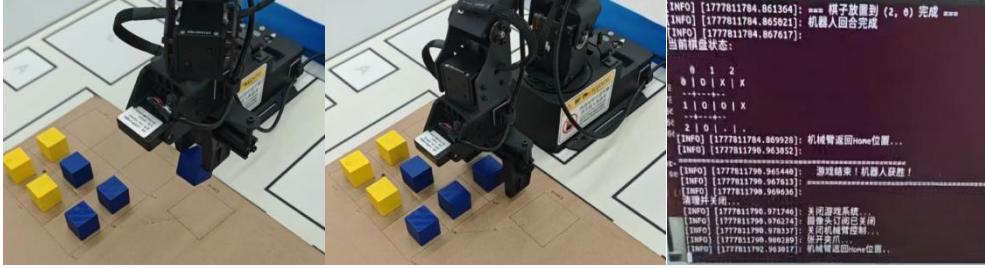


Figure 1: Playing Tic-Tac-Toe with a robotic arm

5. Conclusion

5.1 Final Achievement

We successfully built an LLM-driven robotic arm system that autonomously plays Tic-Tac-Toe against a human opponent. The key achievements are:

- Replacing traditional computer vision (YOLO, color detection) with a VLM (qwen-vl-max) for board state perception. This approach requires zero training data and can be adapted to new board configurations through prompt changes alone.
- Using an LLM (qwen-plus) for game strategy instead of hardcoded algorithms. The LLM demonstrates competent strategic play with interpretable reasoning, and the fallback mechanism ensures robustness.
- Developing a hybrid motion planning strategy (joint-angle + Cartesian), with a three-layer height stratification system that eliminates piece collisions.
- Creating a complete game loop (perceive → reason → act) that runs autonomously from

start to finish, handling board observation, move generation, piece manipulation, and win/draw detection.

5.2 Future Improvements

- **Edge Deployment:** Replace cloud-based LLM/VLM inference with local models running on edge devices (e.g., Qualcomm RB8) to reduce latency from seconds to milliseconds and enable operation without internet connectivity.
- **Extended Game Scenarios:** Extend the system to more complex games (e.g., Connect Four, Gomoku) by updating the VLM prompt and LLM strategy, demonstrating the framework's generality.
- **Code as Policies:** Develop a "Code as Policies" system where the LLM generates complete Python motion scripts rather than single-step JSON commands, enabling more complex multi-step manipulation sequences.
- **Automatic Calibration:** Implement automatic calibration using AR markers (e.g., ArUco tags) on the board, eliminating the manual calibration step and allowing dynamic board repositioning.